

Contents

Chapter — LLM Foundations	2
1. The Fundamental Abstraction	3
2. Tokens Are the Model’s Actual Input Language	4
Why engineers should care	5
3. Subword Tokenization	5
4. Tokenization Is Not Semantically Neutral	6
5. Embeddings: From Tokens to Vectors	6
6. Position Is Part of Meaning	7
7. The Transformer	8
8. Why Attention Matters	8
9. Causal Attention	9
10. From Hidden States to Probabilities	9
11. Generation Is Sampling or Selection	10
12. Temperature	11
13. Top-k and Top-p Sampling	11
Top-k	12
Top-p	12
14. Log Probabilities Are Useful Engineering Signals	12
15. Why Hallucinations Are Possible	13
16. Training Versus Inference	13
Training	13
Inference	14
17. Knowledge Cutoff	14
18. Context Is Externalized Working Memory	15
19. Context Is Not Free	15
20. Context Selection as an Optimization Problem	16
21. The Attention Budget Is Not Uniform	17
22. KV Caching	18
23. Prompt Caching	18
24. Design Prompts Around Stable Prefixes	19
25. Multimodal Models	19
26. When to Use Multimodal Models	20
27. OCR Versus Vision-Language Reasoning	21
28. Reasoning Effort	21
29. Reasoning Is Not the Same as Truth	22
30. Tool Calling Changes the Architecture	22
31. Why Tools Improve Reliability	23
32. Tool Calling Creates a Control Loop	23
33. Structured Outputs	24
34. LLMs Should Not Own Invariants	25
35. Model Selection as an Engineering Problem	25
36. Model Routing	26
37. Mixture-of-Models Architecture	27
38. Fine-Tuning	27

39. Fine-Tuning Versus RAG	28
40. Parameter-Efficient Fine-Tuning	28
41. Quantization	29
42. Quantization Is a Systems Tradeoff	29
43. Self-Hosting	30
44. Throughput Economics	31
45. Reliability Is a System Property	32
46. Verification Changes the Probability Structure	32
47. The Right Mental Model	33
Input	33
Computation	33
Output	33
Properties	33
48. A Practical Capability Matrix	33
49. An Engineering Decision Framework	34
1. What information does the model need?	34
2. Is the information current?	34
3. Is the task deterministic?	34
4. Does the task require another modality?	34
5. How much reasoning is required?	34
6. How expensive is failure?	34
7. Is specialization persistent?	35
50. The Deeper Principle: Move Complexity to the Right Layer	35
51. What You Should Remember	36
Tokenization	36
Transformer inference	36
Sampling	36
Context	36
Caching	36
Knowledge	37
Multimodality	37
Reasoning	37
Tools	37
Fine-tuning	37
Quantization and self-hosting	37
Model selection	37
52. The Core Engineering Mental Model	37

Chapter — LLM Foundations

LLMs are stochastic sequence models embedded inside deterministic software systems.

Engineering with them well requires understanding both sides of that boundary.

A modern large language model can appear deceptively simple from the outside:

prompt $\rightarrow$ model $\rightarrow$ text

But this abstraction hides almost everything that matters to an AI systems engineer.

The model does not fundamentally manipulate words. It manipulates **tokens represented as vectors**, transforms those vectors through a large parameterized computation, and produces a probability distribution over the next token. Everything else—conversation, reasoning, tool use, multimodality, structured output, retrieval, memory, agents—is built around that core mechanism.

Understanding this mechanism changes how you design systems.

It explains why a seemingly trivial prompt can consume a surprising amount of context. It explains why adding more documents can make an answer worse rather than better. It explains why a model can confidently produce an incorrect answer. It explains why a model’s knowledge cutoff matters. It explains why caching can dramatically reduce inference cost. It explains why temperature is not simply a “creativity knob.” It explains why some tasks should use a multimodal model rather than an OCR pipeline, while others should do exactly the opposite.

Most importantly, it gives you a framework for deciding:

**What should the model do, what should the software do,
and what should the model be prevented from doing?**

1. The Fundamental Abstraction

At its mathematical core, an autoregressive language model models a probability distribution over token sequences.

Given tokens

$$x_1, x_2, \dots, x_n,$$

the model factorizes the joint probability as

$$P(x_1, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_1, \dots, x_{t-1}).$$

This is simply the chain rule of probability.

The important consequence is that the model does **not** generate an entire response in one operation.

It repeatedly performs:

$$P(x_{t+1} \mid x_{\leq t}) \rightarrow \text{select } x_{t+1} \rightarrow \text{append token} \rightarrow \text{repeat.}$$

A response of 500 tokens therefore requires approximately 500 sequential decoding decisions.

This distinction is fundamental to understanding:

- latency,
- sampling,
- reasoning,
- context consumption,
- KV caching,
- streaming,
- inference cost,
- tool calls,
- and failure modes.

2. Tokens Are the Model's Actual Input Language

Humans think in words.

LLMs generally do not.

The first stage of inference is **tokenization**.

Consider:

The mortgage payment is \$2,500/month.

A tokenizer might represent this approximately as:

["The", " mortgage", " payment", " is", " \$", "2", "500", "/", "month", "."]

The exact representation depends on the model's tokenizer.

The tokenizer defines a mapping

$$f : \text{text} \rightarrow \{0, \dots, V-1\}^n$$

where V is the vocabulary size and n is the resulting number of tokens.

The reverse operation maps token IDs back into text.

Why engineers should care

Tokenization determines:

1. context-window consumption,
2. inference cost,
3. cache behavior,
4. maximum document size,
5. latency,
6. effective throughput,
7. and sometimes model behavior.

A 100,000-character document is not necessarily “100,000 units” of model input.

The relevant quantity is:

$$N_{\text{tokens}}.$$

3. Subword Tokenization

Modern tokenizers generally use some form of subword segmentation.

The vocabulary contains frequently occurring pieces rather than every possible word.

For example:

engineering

might become:

engine + ering

while a common word might be represented by a single token.

This provides an important compromise.

A vocabulary containing every possible word would be enormous and would have difficulty handling:

- names,
- misspellings,
- source code,
- URLs,
- technical terminology,
- new words,
- multilingual text.

Subword tokenization instead allows the model to represent arbitrary strings using a finite vocabulary.

4. Tokenization Is Not Semantically Neutral

Tokenization creates an important engineering subtlety:

The model’s computational representation does not correspond cleanly to human linguistic units.

This matters especially for:

- numbers,
- source code,
- mathematical notation,
- URLs,
- identifiers,
- unusual names,
- Unicode,
- low-resource languages.

Consider:

123456789

The tokenizer might represent it as several tokens rather than one.

Consequently, the model does not necessarily “see” the number as a single atomic object.

This partly explains why LLMs can struggle with certain forms of exact arithmetic and symbolic manipulation.

The model has learned statistical relationships among token sequences. It is not inherently executing arithmetic on the underlying numerical values.

5. Embeddings: From Tokens to Vectors

Token IDs are discrete integers.

Neural networks operate on continuous numerical representations.

Each token x_i is therefore mapped to an embedding:

$$e_i = E[x_i]$$

where

$$E \in \mathbb{R}^{V \times d}.$$

Here:

- V = vocabulary size,
- d = model embedding dimension.

The sequence becomes a matrix:

$$X = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

At this point, the model has converted discrete symbolic input into a high-dimensional continuous representation.

But token embeddings alone do not tell the model where tokens occur.

The model also needs information about **position**.

6. Position Is Part of Meaning

Compare:

Dog bites man.

with:

Man bites dog.

The same words appear, but their ordering changes the meaning.

Transformers therefore incorporate positional information.

Modern architectures frequently use **rotary positional embeddings (RoPE)** or related mechanisms.

Conceptually, the representation of token i becomes position-dependent:

$$e'_i = R(i)e_i$$

where $R(i)$ is a position-dependent transformation.

The exact mathematics varies by architecture, but the engineering principle is straightforward:

Attention must know not only *which* tokens are present, but *where* they occur.

7. The Transformer

The transformer is the dominant architecture underlying modern LLMs.

Its central operation is **self-attention**.

Given representations X , the model produces:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V.$$

The attention matrix is approximately:

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right).$$

The resulting representation is:

$$Y = AV.$$

This equation deserves careful attention.

The matrix

$$QK^T$$

computes pairwise compatibility between tokens.

The softmax converts these compatibility scores into normalized weights.

The model therefore learns, dynamically, which other tokens should influence the representation of each token.

8. Why Attention Matters

Suppose the input contains:

Alice gave Bob the book because she had finished reading it.

To interpret “she,” the model needs to establish relationships among distant tokens.

Attention provides a mechanism for doing this.

For each position i , the model can construct a weighted combination of information from previous positions:

$$y_i = \sum_j a_{ij} v_j.$$

The weights a_{ij} depend on the learned relationship between the query at i and the keys at j .

This is one reason transformers are so effective at modeling long-range dependencies.

9. Causal Attention

During autoregressive generation, the model cannot see future tokens.

For token t , attention is restricted to:

$$x_1, \dots, x_t.$$

A causal mask enforces this constraint.

The attention matrix therefore has the structure:

$$A_{ij} = 0 \quad \text{for } j > i.$$

This is what allows the same architecture to be trained to predict the next token.

10. From Hidden States to Probabilities

After passing through many transformer layers, the model produces a hidden representation h_t .

A final projection produces logits:

$$z = Wh_t + b.$$

There is one logit for every vocabulary token.

Thus:

$$z \in \mathbb{R}^V.$$

The logits are converted into probabilities using softmax:

$$P(x_{t+1} = i) = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}.$$

The output is therefore not:

“The next token is X.”

It is:

“Here is a probability distribution over possible next tokens.”

That distinction explains much of LLM behavior.

11. Generation Is Sampling or Selection

Suppose the model produces:

Token	Probability
the	0.42
a	0.21
this	0.08
your	0.04
...	...

The system must choose a token.

The simplest approach is greedy decoding:

$$x_{t+1} = \arg \max_i P(x_i \mid x_{\leq t}).$$

But modern systems often sample from the distribution.

That introduces stochasticity.

Consequently:

$$P(y \mid x)$$

is generally not a deterministic function returning one fixed answer. Instead, the system samples a trajectory through a probability distribution.

12. Temperature

Temperature modifies the logits before softmax:

$$P_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

For:

$$T < 1,$$

the distribution becomes sharper.

For:

$$T > 1,$$

the distribution becomes flatter.

As:

$$T \rightarrow 0,$$

sampling approaches greedy selection.

The important engineering point is:

Temperature changes the distribution from which tokens are sampled; it does not directly control intelligence, reasoning ability, or factuality.

Calling temperature a “creativity parameter” is useful as a rough intuition but technically misleading.

13. Top-k and Top-p Sampling

Sampling can also be restricted.

Top-k

Keep only the k highest-probability tokens.

$$S_k = \text{top-}k(P)$$

and renormalize.

Top-p

Choose the smallest set S such that

$$\sum_{i \in S} P_i \geq p.$$

Then sample only from S .

Top-p is adaptive: if the distribution is highly concentrated, the candidate set can be small; if it is diffuse, the set becomes larger.

These parameters are particularly useful when generation diversity matters.

For deterministic business logic, however, relying on sampling behavior is generally inferior to enforcing structure externally.

14. Log Probabilities Are Useful Engineering Signals

Some model APIs expose token-level log probabilities:

$$\log P(x_t \mid x_{<t}).$$

The sequence log-likelihood is:

$$\log P(x_{1:n}) = \sum_{t=1}^n \log P(x_t \mid x_{<t}).$$

The average negative log-likelihood is related to cross-entropy:

$$L = -\frac{1}{n} \sum_{t=1}^n \log P(x_t \mid x_{<t}).$$

Perplexity is:

$$\text{PPL} = e^L.$$

These quantities are fundamental in language-model training and evaluation.

However, engineers should be careful about interpreting token probabilities as **confidence**.

A high probability for a token sequence does not imply that the underlying proposition is true.

A model can assign high probability to a fluent false statement.

15. Why Hallucinations Are Possible

Consider a request:

Who invented technology X in 1847?

If the model has no reliable knowledge of X, it still has to produce a probability distribution over tokens.

The model's objective is not:

produce a true statement.

Its training objective is much closer to:

$$\max_{\theta} \sum_t \log P_{\theta}(x_t \mid x_{<t}).$$

This rewards predicting plausible continuations.

Truth is correlated with linguistic patterns in the training data, but truth is not identical to likelihood.

This distinction is one of the central facts of LLM engineering.

16. Training Versus Inference

A model has two very different phases.

Training

Training adjusts parameters:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L.$$

The model learns statistical structure from enormous datasets.

Inference

During inference, the parameters are normally fixed.

The model computes:

$$P_{\theta}(x_{t+1} \mid x_{\leq t})$$

using those parameters.

Inference therefore does not normally “teach” the model something permanently.

If you tell a model:

Remember that our database schema uses `customer_uuid`.

the information exists in the current context, not necessarily in the model’s parameters.

This distinction becomes critical when designing memory systems.

17. Knowledge Cutoff

A pretrained model has a finite training corpus.

Suppose training effectively ends at time T .

Information created after T cannot be directly present in the pretrained parameters unless it entered through some subsequent update.

Thus:

model knowledge $\neq$ current world state.

This is why systems that depend on current information need mechanisms such as:

- web search,
- retrieval,
- APIs,
- databases,
- tool calls,
- or continuously updated model versions.

A knowledge cutoff is therefore not merely a documentation detail.

It is an architectural constraint.

18. Context Is Externalized Working Memory

The context window provides information that can be used during inference without changing the model’s parameters.

Conceptually:

$$P_{\theta}(y \mid x, \mathcal{C})$$

where $\mathcal{C}$ is the supplied context.

This makes context one of the most important engineering resources in an LLM system.

It can contain:

- system instructions,
- conversation history,
- retrieved documents,
- tool results,
- examples,
- schemas,
- source code,
- images,
- structured data.

The context window is therefore analogous to a constrained working-memory budget.

19. Context Is Not Free

Suppose:

$$N_{\text{in}}$$

is the number of input tokens and

$$N_{\text{out}}$$

is the number of generated tokens.

A simple inference-cost model is:

$$C \approx c_{\text{in}}N_{\text{in}} + c_{\text{out}}N_{\text{out}}.$$

The exact pricing and computational characteristics vary by model, but the engineering principle is universal:

More context has a cost.

There is also a latency cost.

And there is an information-quality cost.

A larger context can contain more relevant information, but it can also contain:

- irrelevant information,
- contradictory information,
- stale information,
- duplicated information,
- distracting information.

Therefore:

more context $\nRightarrow$ better answer.

20. Context Selection as an Optimization Problem

Suppose you have candidate context elements:

$$C_1, C_2, \dots, C_n.$$

Each has:

- relevance r_i ,
- token cost c_i ,
- reliability q_i .

You can conceptualize context selection as:

$$\max_S \sum_{i \in S} U(C_i)$$

subject to:

$$\sum_{i \in S} c_i \leq B$$

where B is the context budget.

This resembles a knapsack problem.

In real systems, utility might be approximated by:

$$U(C_i) = \alpha r_i + \beta q_i - \gamma c_i.$$

This gives a useful engineering mindset:

Retrieval is not simply “find documents.” It is **allocate scarce context to information with the highest expected utility.**

21. The Attention Budget Is Not Uniform

A model with a large context window does not necessarily use every token equally effectively.

Information can be:

- near the beginning,
- near the end,
- buried in the middle,
- repeated,
- surrounded by distractors.

Empirical behavior often exhibits position sensitivity and “lost in the middle” effects.

Therefore, context engineering should consider not only:

$$N_{\text{tokens}}$$

but also:

information density

and

information placement.

This is one reason concise structured context can outperform dumping an entire document into a prompt.

22. KV Caching

Autoregressive generation repeatedly processes the growing prefix.

Naively, this would require recomputing attention-related representations for the entire prefix at every generation step.

Transformer inference therefore uses a **key-value cache**.

For previous tokens, the model stores:

$$K_1, \dots, K_t$$

and

$$V_1, \dots, V_t.$$

When generating token $t + 1$, the model only needs to compute the new query and append the new key/value states.

This dramatically improves decoding efficiency.

The important distinction is:

KV cache is computational state, not semantic memory.

It accelerates inference for an existing context.

It does not mean the model has learned the information permanently.

23. Prompt Caching

A higher-level optimization is prompt or prefix caching.

Suppose thousands of requests share:

```
System instructions
+
company policy
+
large schema
```

Only the final user query changes.

If the provider supports prefix caching, the shared prefix can be reused.

Conceptually:

$$\text{request} = \underbrace{P}_{\text{stable prefix}} + \underbrace{Q}_{\text{dynamic suffix}}.$$

The system can cache computation associated with P .

This can reduce:

- latency,
- compute,
- and sometimes cost.

Therefore, prompt architecture can have direct economic consequences.

24. Design Prompts Around Stable Prefixes

This leads to an important systems principle:

**Put stable information early and volatile information late
when the inference stack benefits from prefix caching.**

For example:

SYSTEM

POLICY

TOOLS

SCHEMA

EXAMPLES

USER REQUEST

is often more cache-friendly than continuously reconstructing the entire prompt in a different order.

This is not merely prompt-writing advice.

It is **inference architecture**.

25. Multimodal Models

Text-only models operate on tokenized text.

Multimodal systems can process other modalities:

- images,
- audio,
- video,
- documents,
- potentially additional sensor modalities.

An image is not naturally represented as a sequence of words.

A multimodal architecture therefore needs an encoding mechanism that converts the modality into representations usable by the model.

Conceptually:

$$I \rightarrow E_{\text{vision}}(I) \rightarrow Z \rightarrow \text{language model}.$$

The visual encoder might produce a sequence of embeddings:

$$Z \in \mathbb{R}^{m \times d}.$$

These representations can then interact with language representations.

26. When to Use Multimodal Models

Use a multimodal model when the semantics of the task depend on information that is difficult or lossy to convert into text.

Examples include:

- interpreting charts,
- analyzing photographs,
- reading diagrams,
- understanding screenshots,
- inspecting UI layouts,
- extracting relationships from documents,
- analyzing handwritten content.

But multimodal does not automatically mean better.

For a purely textual task, introducing images can increase:

- inference cost,
- latency,
- complexity,
- and potential failure modes.

The correct question is:

Does the additional modality contain information that materially improves the decision?

27. OCR Versus Vision-Language Reasoning

Consider a scanned invoice.

If the goal is:

Extract the invoice number.

A specialized OCR system may be preferable.

If the goal is:

Determine whether the invoice’s shipping address matches the address in the purchase order and explain discrepancies.

A multimodal reasoning model may be much more useful.

This is an example of a general architecture principle:

Use the least expensive representation that preserves the information required by the task.

28. Reasoning Effort

Modern models increasingly expose some form of reasoning-effort control.

Conceptually, we can view inference as having a computational budget:

$$B_r.$$

A harder problem may benefit from allocating more computation:

$$B_r \uparrow \Rightarrow \text{potentially stronger reasoning.}$$

But:

$$B_r \uparrow \Rightarrow \text{latency} \uparrow$$

and often:

$$\text{cost} \uparrow .$$

Therefore reasoning effort is another resource-allocation decision.

You should not automatically use maximum reasoning for every request.

A useful routing strategy is:

difficulty $\rightarrow$ reasoning budget.

For example:

Task	Reasoning budget
Classification	Low
Simple extraction	Low
Summarization	Low–medium
Coding	Medium–high
Architecture design	High
Complex mathematical proof	High
Multi-step research	High

29. Reasoning Is Not the Same as Truth

A model can spend more computation reasoning about a false premise.

Therefore:

more reasoning $\nRightarrow$ guaranteed correctness.

Reasoning effort increases the opportunity to solve difficult problems; it does not provide a formal correctness guarantee.

For high-assurance systems, external verification remains essential.

30. Tool Calling Changes the Architecture

A plain LLM produces text.

A tool-enabled LLM can instead produce a structured request:

```
{
  "tool": "get_weather",
  "arguments": {
    "location": "Portland, Oregon"
  }
}
```

The surrounding system executes the operation and returns the result to the model.

The interaction becomes:

LLM $\rightarrow$ tool request $\rightarrow$ deterministic system $\rightarrow$ tool result $\rightarrow$ LLM.

This is one of the most important architectural transitions in modern AI systems.

31. Why Tools Improve Reliability

Suppose the model needs the current exchange rate.

Without a tool:

model parameters $\rightarrow$ guess.

With a tool:

model $\rightarrow$ FX API $\rightarrow$ current rate.

The model is no longer responsible for remembering volatile information.

Similarly:

Problem	Better mechanism
Current weather	Weather API
Current stock price	Market-data API
Database query	SQL/database tool
Arithmetic	Calculator/code
Current documentation	Retrieval/search
File manipulation	File-system tool
Sending email	Email tool

This produces a fundamental design rule:

Do not ask the model to simulate a deterministic capability that the system can provide directly.

32. Tool Calling Creates a Control Loop

An agentic system can be modeled as:

$$s_t \xrightarrow{\text{LLM}} a_t \xrightarrow{\text{environment}} o_{t+1} \xrightarrow{\text{LLM}} a_{t+1}.$$

Where:

- s_t = current state,
- a_t = action,
- o_t = observation.

This is much closer to a control system than to ordinary text generation.

The engineering problems therefore expand to include:

- action validation,
- authorization,
- retries,
- timeouts,
- idempotency,
- state management,
- observability,
- sandboxing,
- rollback.

33. Structured Outputs

If software must consume model output, free-form text is usually the wrong interface.

Instead of:

The customer appears to be high priority...

request a schema such as:

```
{  
  "priority": "high",  
  "confidence": 0.87,  
  "reason": "..."  
}
```

The model still generates tokens, but constrained decoding or structured-output mechanisms can reduce the space of valid outputs.

Conceptually, instead of:

$$x_{t+1} \in V,$$

we constrain generation to:

$$x_{t+1} \in V_t^{\text{valid}}.$$

This moves part of the correctness burden from probabilistic generation into deterministic validation.

34. LLMs Should Not Own Invariants

Suppose a financial application requires:

$$\text{balance} \geq 0.$$

Do not merely instruct the LLM:

Never produce a negative balance.

Enforce the invariant in software.

Likewise:

- authorization should be deterministic,
- monetary calculations should be deterministic,
- database constraints should be deterministic,
- security policies should be deterministic,
- transaction semantics should be deterministic.

The LLM should operate **inside the invariant envelope**.

35. Model Selection as an Engineering Problem

There is rarely a single universally optimal model.

Suppose you have models:

$$M_1, M_2, \dots, M_k.$$

Each has characteristics:

$$M_i = (Q_i, L_i, C_i, R_i, S_i)$$

where:

- Q = quality,
- L = latency,
- C = cost,
- R = reliability,
- S = capability/scope.

Model selection is therefore a constrained optimization problem.

For example:

$$\max_i Q_i$$

subject to:

$$L_i < L_{\max}$$

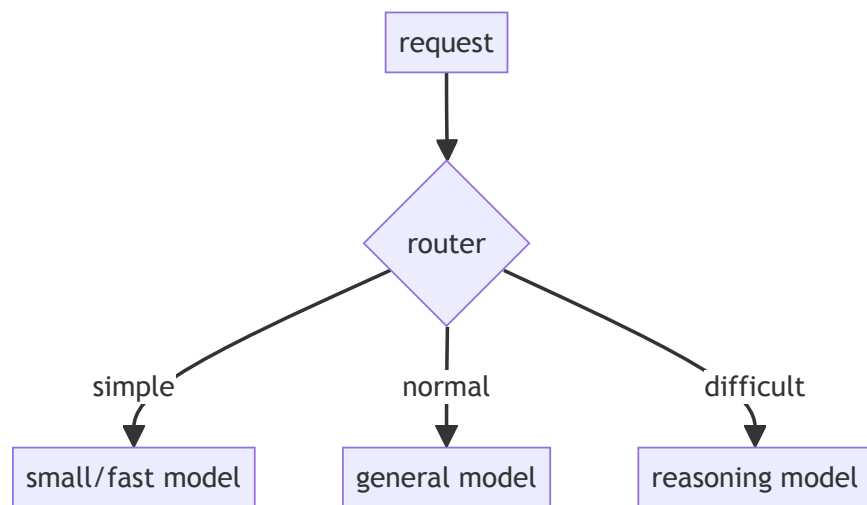
$$C_i < C_{\max}.$$

In practice, the objective is multi-dimensional.

36. Model Routing

Rather than choosing one model for everything, route requests.

For example:



The router itself can be:

- deterministic rules,
- a classifier,
- a small LLM,
- a larger LLM,

- or a hybrid.

This architecture can dramatically improve the quality/cost frontier.

37. Mixture-of-Models Architecture

A production system might use:

$$M_{\text{small}} \rightarrow M_{\text{general}} \rightarrow M_{\text{reasoning}}$$

depending on task complexity.

For example:

- small model for intent classification,
- embedding model for retrieval,
- vision model for image extraction,
- general LLM for response generation,
- reasoning model for difficult analysis,
- deterministic code for calculation,
- external APIs for current data.

This is often more effective than asking one enormous model to perform every function.

38. Fine-Tuning

Prompting changes the **input**.

Fine-tuning changes the **parameters**.

Given a pretrained model:

$$\theta_0,$$

fine-tuning optimizes:

$$\theta^* = \arg \min_{\theta} L_{\text{task}}(\theta).$$

The goal is to alter model behavior for a particular distribution or task.

Fine-tuning is useful when you need persistent changes in:

- style,
- formatting,

- classification behavior,
- domain-specific patterns,
- instruction following,
- specialized task performance.

It is generally **not** the first tool to use for frequently changing factual knowledge.

39. Fine-Tuning Versus RAG

Consider a company handbook.

If the handbook changes every week, putting its contents into model parameters is operationally awkward.

RAG is more appropriate:

query $\rightarrow$ retrieve current information $\rightarrow$ LLM.

Fine-tuning is more appropriate for persistent behavior:

input $\rightarrow$ specialized model behavior.

A useful rule is:

Use retrieval to change what the model knows at inference time; use fine-tuning to change how the model behaves.

The distinction is not absolute, but it is an excellent architectural default.

40. Parameter-Efficient Fine-Tuning

Full fine-tuning can require updating billions of parameters.

Parameter-efficient approaches such as LoRA instead learn a relatively small update.

A conceptual formulation is:

$$W' = W + \Delta W$$

with

$$\Delta W = BA$$

where the rank of the update is much smaller than the dimensions of W .

This can dramatically reduce:

- memory requirements,
- training cost,
- storage,
- deployment complexity.

The base model remains largely unchanged.

41. Quantization

Model parameters are normally stored in floating-point representations.

Quantization reduces representation precision.

For example:

$$FP16 \rightarrow INT8$$

or:

$$FP16 \rightarrow INT4.$$

Conceptually, a floating-point weight w is mapped to a lower-precision representation:

$$q = \text{round} \left(\frac{w}{s} \right)$$

with a scale factor s .

The reconstructed approximation is:

$$\hat{w} = sq.$$

The objective is to reduce memory and improve inference efficiency while keeping accuracy degradation acceptable.

42. Quantization Is a Systems Tradeoff

Suppose a model has N parameters.

At approximately b bits per parameter, the raw parameter memory is:

$$M \approx \frac{Nb}{8}.$$

A 70-billion-parameter model at 16 bits requires approximately:

$$70 \times 10^9 \times 2 \approx 140 \text{ GB}$$

just for the raw weights, before additional runtime memory.

At 4 bits:

$$70 \times 10^9 \times 0.5 \approx 35 \text{ GB}.$$

This is why quantization can transform which hardware is capable of hosting a model.

But inference memory also includes:

- KV cache,
- activations,
- runtime overhead,
- batching,
- framework-specific allocations.

Therefore:

$$\text{model size} \neq \text{total inference memory}.$$

43. Self-Hosting

Self-hosting means operating the model infrastructure yourself rather than using a hosted inference API.

Potential benefits include:

- data locality,
- predictable availability,
- customization,
- control over model versions,
- potentially lower marginal cost at high utilization.

Costs include:

- hardware,
- electricity,
- model serving,

- monitoring,
- upgrades,
- capacity planning,
- security,
- operational expertise.

The relevant comparison is not:

API price vs GPU price.

It is:

total cost of ownership.

44. Throughput Economics

Suppose a self-hosted system costs:

$$C_{\text{fixed}}$$

per month and serves:

$$N$$

requests.

Its approximate fixed cost per request is:

$$C_{\text{request}} = \frac{C_{\text{fixed}}}{N}.$$

As utilization increases, this can become attractive.

But at low utilization, the infrastructure sits idle.

Therefore self-hosting economics depend strongly on:

utilization.

This is why “which model is cheapest?” is often the wrong question.

The real question is:

**What is the cost per successful task at the required quality,
latency, reliability, and utilization?**

45. Reliability Is a System Property

A model can have excellent benchmark scores and still produce an unreliable application.

Suppose a workflow contains three probabilistic steps with success probabilities:

$$p_1, p_2, p_3.$$

If all three must succeed, then approximately:

$$P(\text{success}) = p_1 p_2 p_3.$$

If:

$$p_1 = p_2 = p_3 = 0.95,$$

then:

$$P(\text{success}) = 0.95^3 \approx 0.857.$$

Three individually strong components can therefore produce a significantly weaker end-to-end system.

This is why AI engineering cannot be reduced to model selection.

46. Verification Changes the Probability Structure

Suppose the model produces an answer and a deterministic verifier catches errors.

Then the system becomes:

$$\text{generate} \rightarrow \text{verify} \rightarrow \text{accept/reject}.$$

This can substantially improve reliability.

Examples:

- generated SQL $\rightarrow$ parser,
- generated JSON $\rightarrow$ schema validator,
- generated code $\rightarrow$ compiler + tests,
- mathematical answer $\rightarrow$ calculator,
- API action $\rightarrow$ authorization layer,
- retrieved answer $\rightarrow$ citation validation.

The system should exploit deterministic computation wherever possible.

47. The Right Mental Model

An LLM is best understood as a component with a peculiar contract:

Input

A sequence of multimodal representations containing instructions, data, examples, and state.

Computation

A large learned function producing a conditional probability distribution.

Output

A probabilistically generated sequence or structured action.

Properties

- powerful,
- flexible,
- approximate,
- probabilistic,
- context-sensitive,
- expensive,
- difficult to formally verify.

This leads to an engineering philosophy:

Treat the LLM as a probabilistic subsystem, not as an oracle.

48. A Practical Capability Matrix

When designing an AI system, ask which mechanism should perform each operation.

Requirement	Preferred mechanism
Natural-language interpretation	LLM
Classification	LLM / classifier
Current information	Retrieval/API
Exact arithmetic	Code
Database lookup	Database

Requirement	Preferred mechanism
Authorization	Deterministic policy
Structured transformation	LLM + schema
Image understanding	Vision-language model
Speech transcription	Speech model
Semantic similarity	Embedding model
Persistent behavioral specialization	Fine-tuning
Large-scale private inference	Potentially self-hosted model
Deterministic validation	Conventional software
Complex planning	Reasoning-capable LLM
External action	Tool calling + policy layer

The best AI systems are therefore usually **hybrid systems**.

49. An Engineering Decision Framework

For every proposed LLM feature, ask seven questions.

1. What information does the model need?

Determine:

$$C_{\text{required}}$$

Do not automatically provide everything available.

2. Is the information current?

If yes, consider retrieval or tools.

3. Is the task deterministic?

If yes, consider conventional software instead of generation.

4. Does the task require another modality?

If yes, use a multimodal model or specialized modality model.

5. How much reasoning is required?

Allocate an appropriate reasoning budget.

6. How expensive is failure?

The higher the cost, the more external verification you need.

7. Is specialization persistent?

If yes, consider fine-tuning.

This gives you a simple architecture-selection function:

$$A^* = \arg \max_A [Q(A) - \lambda C(A) - \mu L(A) - \nu R_{\text{failure}}(A)].$$

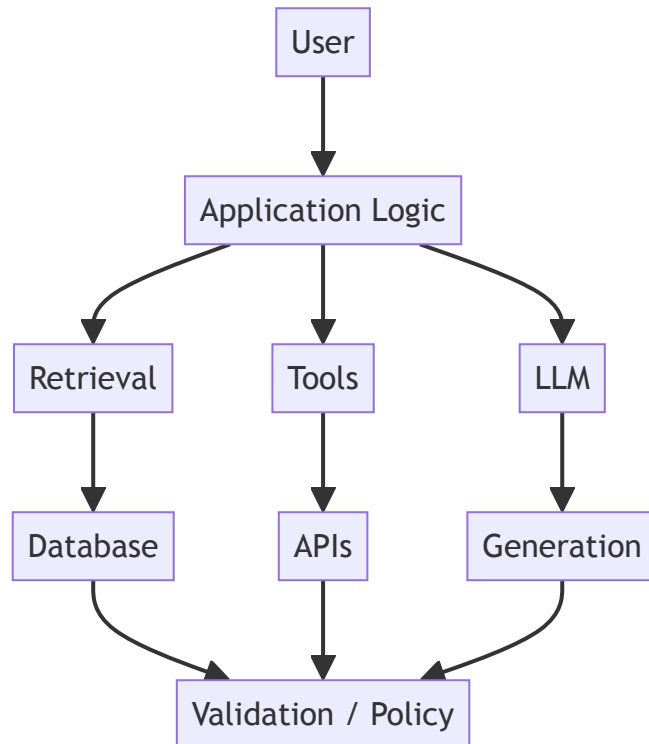
The coefficients reflect the business requirements.

50. The Deeper Principle: Move Complexity to the Right Layer

A mature AI system does not attempt to make the LLM solve every problem.

Instead, it assigns each problem to the layer best suited to solve it.

For example:



The LLM supplies what it is uniquely good at:

mapping ambiguous human language and complex information into useful probabilistic decisions.

The surrounding software supplies:

determinism, memory, authority, verification, state, and control.

51. What You Should Remember

The mathematical foundations lead directly to practical engineering conclusions.

Tokenization

The model operates on tokens, not words.

Therefore token count—not character count—is the fundamental unit of context and inference.

Transformer inference

The model computes a conditional probability distribution:

$$P(x_{t+1} \mid x_{\leq t}).$$

It does not retrieve a deterministic answer from a database of facts.

Sampling

Temperature, top-k, and top-p alter the generation distribution.

They do not create intelligence or guarantee correctness.

Context

Context is external working memory.

It has finite capacity and economic cost.

More context can improve or degrade performance.

Caching

KV caching accelerates autoregressive decoding.

Prompt/prefix caching can make stable context dramatically cheaper.

Knowledge

Model parameters represent learned statistical structure, not a live database of the world.

Current information should generally come from external sources.

Multimodality

Use multimodal models when non-text information carries important semantics.

Otherwise, a simpler specialized pipeline may be better.

Reasoning

Reasoning effort is a computational resource.

Allocate it according to task difficulty and failure cost.

Tools

Use tools whenever correctness depends on deterministic, current, or externally authoritative information.

Fine-tuning

Fine-tuning changes persistent model behavior.

Retrieval changes the information available at inference time.

Quantization and self-hosting

These are systems and economics decisions, not merely model decisions.

Model selection

The best model is not necessarily the largest or most capable one.

The right model is the one that optimizes the complete system objective.

52. The Core Engineering Mental Model

The most useful equation in this chapter may not be a transformer equation.

It is this:

$\text{AI System} = \text{Probabilistic Model} + \text{Context} + \text{Tools} + \text{Deterministic Software} + \text{Verification}$

The LLM is the probabilistic core.

Everything around it exists to make that probabilistic core useful, bounded, economical, observable, and reliable.

That distinction marks the transition from **using an LLM** to **engineering an AI system**.

And it is the foundation for everything that follows: context engineering, RAG, evaluations, agentic workflows, production architecture, model routing, coding agents, and ultimately the design of reliable AI-native software.